



武汉大学

计算机学院

《大数据与云计算技术》

本科生课程实验报告

题 目 基于 openGauss 的云资源成本与碳排放分析平台设计与实现

专 业 特色软件工程

班 级 特软班

姓 名 童济舟

学 号 2024302111261

武汉大学计算机学院

2026 年 6 月 20 日

目 录

1 概述	1
1.1 课程选题背景（或需求分析）	1
1.2 课程实验内容	1
2 系统设计与实现	4
2.1 系统架构设计	4
2.2 系统技术选型	4
2.3 数据库设计	5
2.4 功能模块设计与实现	7
2.4.1 资源管理模块	7
2.4.2 用量采集与成本估算模块	8
2.4.3 分析总览模块	9
2.4.4 成本分析模块	10
2.4.5 优化模拟模块	11
2.4.6 告警中心模块	11
2.4.7 CSV 导入、导出和周报模块	12
2.4.8 Spark 批处理聚合模块	12
2.5 关键实现说明	13
2.5.1 后端启动自动建表和初始化数据	13
2.5.2 openGauss 与 SQLAlchemy 兼容处理	13
2.5.3 前端组件拆分	13
3 应用部署	15
3.1 实验环境	15
3.2 部署 openGauss / GaussDB	15
3.3 部署后端应用	18
3.4 部署前端应用	19
3.5 Spark 批处理任务	20
4 测试评估	23
4.1 测试方法	23
4.2 功能测试结果	23

4.3	数据库读写验证	24
4.4	部署验证	25
5	总结	26
5.1	项目开发的挑战与应对方法	26
5.2	项目部署存在的问题与不足	26
5.3	项目展望与学习心得	26
6	参考文献	28

1 概述

1.1 课程选题背景（或需求分析）

本次课程实验要求部署 openGauss / GaussDB 数据库，并完成一个前后端分离的 Web 应用。最开始设计题目时，我没有选择只做一个简单的信息管理系统，而是结合云计算课程内容，把应用主题定为“云资源成本与碳排放分析平台”。主要原因是因为我的华为云预算控制没做好造成欠费了——因此至少对我而言这是十分有需求的。并且，除了预算，使用也和排放息息相关。这个题目和云平台的实际使用场景比较接近：在一台云主机、一个数据库或者一个对象存储桶刚创建时，资源数量不多，人工记录还可以接受；但资源数量增加后，哪些资源成本高、哪些资源利用率低、哪些资源接近预算，靠表格很难及时发现。

因此，本实验的核心需求可以分为三类。

第一类是课程任务书中的基础要求：需要在容器中部署 openGauss / GaussDB，后端能够读写数据库，前端和后端分别制作 Docker 镜像，并在华为云开发者空间中完成部署和演示。

第二类是 Web 应用本身的业务需求：需要能够录入云资源、写入用量数据、展示统计图表，并支持查询、删除、导出等基本操作。为了证明数据不是写死在前端页面中，所有页面数据都应通过后端接口从数据库读取。

第三类是为了让项目更完整而增加的分析类需求：系统需要根据用量数据估算成本和碳排放，识别预算风险和异常成本，并输出可执行的优化建议。这样在演示时可以体现出 openGauss 中的数据经过后端计算后，对前端展示和运维决策产生了影响。

本项目最终实现了一个名为 CloudCostLab 的云资源成本与碳排放分析平台。平台支持资源管理、用量采集、成本估算、碳排放估算、预算风险分析、成本预测、异常检测、告警确认、CSV 导入、周报导出和 Spark 批处理聚合。和普通 CRUD 项目相比，本项目更强调数据从“录入 - 入库 - 聚合 - 分析 - 展示 - 报告”的完整流程。

1.2 课程实验内容

根据课程任务书，本实验需要完成以下内容：

1. 使用容器化技术在华为云开发者空间部署伪分布式 openGauss /

GaussDB 数据库服务。

2. 使用 Java 或 Python 开发前后端分离的 Web 应用。
3. Web 应用需要有完整功能和界面，并且前端 UI 元素要有对应后台处理逻辑。
4. 后端需要支持对 openGauss / GaussDB 的读写。
5. 建议使用 Spark、Flink 等大数据处理框架对 openGauss / GaussDB 进行读写。
6. 前端和后端都要通过 Dockerfile 制作容器镜像，并在华为云开发者空间部署、测试和总结。

本项目对应完成情况如下。

任务书要求	本项目完成情况
部署 openGauss / GaussDB	通过 Docker Compose 启动 openGauss 容器，容器内端口 5432，宿主机端口 15432
前后端分离 Web 应用	前端为 React + Vite，后端为 FastAPI，前端通过 /api 访问后端
前端 UI 有后台逻辑	新增、写入用量、查询、状态切换、删除、聚合、分析、告警、导入、导出等按钮均对应后端接口
后端读写数据库	使用 SQLAlchemy + psycopg2 连接 openGauss，读写 6 张业务表
Dockerfile 制作镜像	backend/Dockerfile、frontend/Dockerfile、spark-job/Dockerfile 均已编写
大数据框架	使用 Spark JDBC 读取 openGauss 明细数据并写回聚合快照
部署与测试文档	编写 README、部署指南、演示脚本、API 示例和本实验报告

项目目录结构如下。

```
cloudcompute/
```

《大数据与云计算技术》本科生课程实验报告

└── backend/	FastAPI 后端
└── frontend/	React 前端
└── spark-job/	Spark JDBC 聚合任务
└── database/	表结构 SQL
└── docs/	部署说明、演示脚本、报告、截图和课程任务书
└── latex-report/	LaTeX 报告源码和模板依赖
└── samples/	CSV 导入样例
└── scripts/	部署前自检脚本
└── docker-compose.yml	容器编排文件

源码除了在附件里有，也提供在 GitHub 上，地址为 <https://github.com/fQwQf/cloudcompute>。部署方法详见 README。

2 系统设计与实现

2.1 系统架构设计

系统采用前后端分离架构，并用 Docker Compose 将前端、后端、数据库和 Spark 任务组织在同一个容器网络中。浏览器访问前端容器，前端容器中的 Nginx 提供静态页面，并将 /api 请求反向代理到后端容器。后端容器运行 FastAPI 服务，使用 SQLAlchemy ORM 访问 openGauss。Spark 容器不是常驻服务，而是作为批处理任务按需运行，通过 JDBC 读取 openGauss 数据并写回聚合快照。

系统整体结构如下。

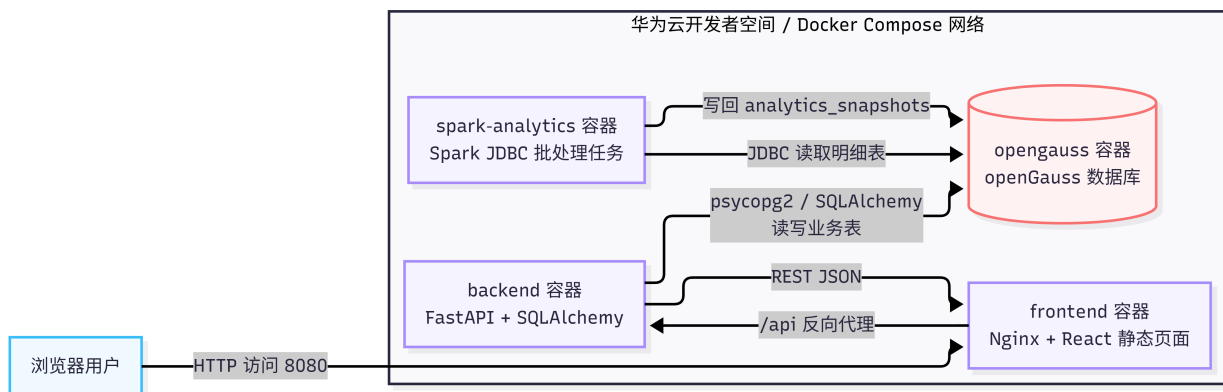


图 2.1 系统总体架构图

数据流转过程如下：

1. 用户在前端页面提交资源或用量表单。
2. 前端调用 FastAPI 后端接口。
3. 后端校验请求参数，并通过 SQLAlchemy 写入 openGauss。
4. 后端在服务层计算成本、碳排放、趋势、预算风险和告警。
5. 前端重新请求接口，展示最新指标、图表、告警和审计日志。
6. Spark 任务通过 JDBC 读取明细表，批量生成分析快照并写回 openGauss。

这个架构的优点是边界比较清晰：前端只负责交互和展示，后端负责业务逻辑和数据库访问，openGauss 负责持久化数据，Spark 负责批处理分析。检查时也可以分别展示页面、接口文档、数据库表和容器状态。

2.2 系统技术选型

本项目主要技术选型如下。

层次	技术	使用原因
数据库	openGauss	满足课程要求，支持 SQL 查询和容器部署
后端	FastAPI	Python 编写方便，接口文档自动生成，适合课程演示
ORM	SQLAlchemy	通过模型定义表结构，避免手写大量 SQL
数据库驱动	psycopg2	可通过 PostgreSQL 协议连接 openGauss
前端	React + TypeScript + Vite	组件化开发，适合仪表盘、表格和表单
图表	Recharts	实现趋势图、柱状图和预测图
图标	lucide-react	用于按钮和指标卡，界面更清晰
Web 服务	Nginx	提供前端静态文件，并代理 /api
容器	Docker + Docker Compose	满足课程容器化部署要求
批处理	Spark	作为大数据框架，通过 JDBC 读写 openGauss

在部署环境选择上，项目默认使用国内镜像站，避免直接访问 Docker Hub 超时。后端 Python 依赖使用阿里云 PyPI 源，前端 npm 依赖使用 npmmirror，Spark JDBC 驱动使用阿里云 Maven 仓库下载。

2.3 数据库设计

数据库设计时，我尽量把“资源基础信息”“资源用量明细”“分析结果”“告警结果”和“操作审计”分开保存，便于后端分别查询和聚合。系统共使用 6 张主要表。

表名	主要作用
cloud_resources	保存云资源基础信息，如名称、区域、类型、负责人、状态和月预算
usage_records	保存每日用量明细，如 CPU 核时、内存 GB 时、存储 GB 时、网络流量、成本和碳排放
analytics_snapshots	保存统计快照，如资源数量、总成本、平均利用率、风险数量
cloud_insights	保存成本分析建议，如降配建议、预算风险、异常成本和预计节省
cloud_alerts	保存告警信息，如预算告警、异常成本告警、利用率告警和碳排放告警
audit_events	保存审计日志，记录新增、更新、删除、导入、分析、确认告警等操作

核心表字段说明如下。

cloud_resources 表记录资源的基础属性，其中 monthly_budget 用于后续计算预算风险，status 用于区分 active、paused、retired 三种状态。

usage_records 表记录资源每日用量。后端在写入时会自动计算 estimated_cost 和 carbon_kg，因此前端不需要自己计算费用。

analytics_snapshots 表保存聚合快照。该表有一个 generated_by 字段，用于区分快照是由 API 生成还是由 Spark 任务生成。

cloud_alerts 表保存告警规则执行结果，并通过 status 和 acknowledged_at 字段记录告警是否已确认。

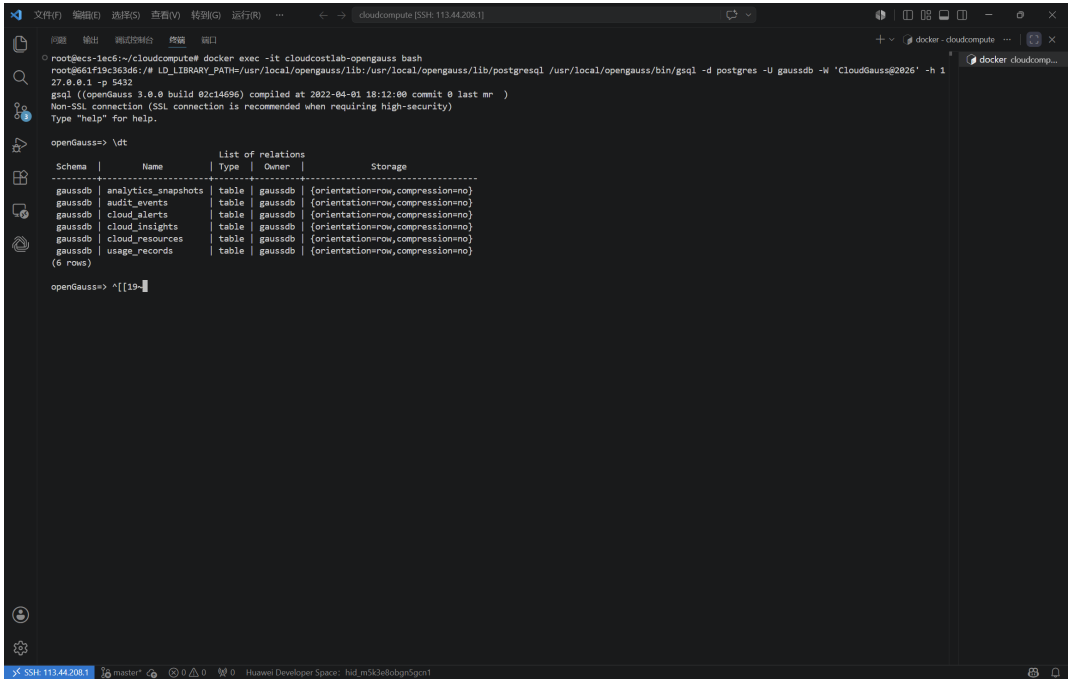


图 2.2 数据库表结构截图

2.4 功能模块设计与实现

2.4.1 资源管理模块

资源管理模块实现资源新增、查询、状态切换和删除。前端“新增资源”表单包含名称、区域、类型、负责人和月预算。资源表格支持按名称、负责人和类型查询，也支持按状态过滤。

对应后端接口如下。

接口	功能
GET /api/resources	查询资源列表
POST /api/resources	新增资源
GET /api/resources/{resource_id}	查询单个资源
PUT /api/resources/{resource_id}	修改资源信息或状态
DELETE /api/resources/{resource_id}	删除资源

每次新增、修改或删除资源时，后端都会写入一条审计日志。这样在演示时可以同时展示“前端操作成功”和“数据库中留下了操作记录”。

新增资源

名称

ecs-prod-api

区域

cn-east-3

类型

ECS

负责人

平台组

月预算

500

+ 写入资源

写入用量

资源

sales-report

日期

06/20/2026

CPU 核时

24

内存 GB时

96

存储 GB时

500

网络 GB

20

利用率

70

写入用量

图 2.3 新增资源与写入用量表单

资源清单						
Q 名称、负责人、类型 全部状态 Q 查询						
名称	区域	类型	负责人	状态	预算	操作
sales-report	cn-east-3	MRS/Spark	数据组	active	¥760	
audit-archive	cn-south-1	OBS	运维组	active	¥240	
customer-ledger	cn-north-4	GaussDB	数据组	active	¥880	
order-service	cn-east-3	ECS	平台组	active	¥520	

图 2.4 资源清单、查询和状态操作

2.4.2 用量采集与成本估算模块

用量采集模块用于向数据库写入每日资源用量。前端输入 CPU 核时、内存 GB 时、存储 GB 时、网络 GB 和利用率，后端在 `create_usage_record` 服务函数中完成成本和碳排放估算。

成本估算公式如下。

```
estimated_cost =  
    cpu_core_hours * 0.045  
    + memory_gb_hours * 0.012  
    + storage_gb_hours * 0.0006  
    + network_gb * 0.08
```

碳排放估算公式如下。

```
energy_kwh =  
    cpu_core_hours * 0.035
```

```
+ memory_gb_hours * 0.004  
  
+ storage_gb_hours * 0.0002  
  
carbon_kg = energy_kwh * 0.581
```

这里的费用和碳排放是课程项目中的估算模型，目的是让系统有统一的统计口径。真实生产环境中应接入云厂商账单和能耗数据。

2.4.3 分析总览模块

分析总览模块通过 GET /api/analytics/overview?days=30 接口返回近 30 天资源成本、碳排放、利用率和预算风险。前端首页展示以下内容：

- 资源总数和活跃资源数。
- 近 30 天成本和碳排放。
- 预算风险数量。
- 节省潜力和未确认告警数。
- 近 30 天成本趋势图。
- 资源成本排行。
- 预算风险列表。

点击“聚合”按钮后，前端调用 POST /api/analytics/recompute，后端将当前统计结果保存到 analytics_snapshots 表中。这个功能用于证明分析结果不仅在内存中计算，还可以写回 openGauss。



图 2.5 系统总览页面

2.4.4 成本分析模块

成本分析模块是本项目的主要扩展功能。后端会读取近 30 天 `usage_records`，然后生成以下结果：

1. 未来 7 天成本预测。
2. 未来 30 天成本和碳排放估算。
3. 总预算压力。
4. 可节省金额估算。
5. 低利用率资源降配建议。
6. 预算接近上限的资源提醒。
7. 异常成本尖峰检测。
8. 高碳排放资源提示。

前端点击“分析”按钮时，会调用 `POST /api/insights/recompute`，后端重新生成分析结果并写入 `cloud_insights` 表。页面中的“优化建议”和“异常成本尖峰”都来自这个接口。

异常检测的实现思路是：按资源和日期聚合成本，计算某个资源近 30 天的平均成本和标准差，如果某一天成本明显高于基线，则认为存在成本尖峰。这个方法不是复杂模型，但对于课程项目来说足够展示数据分析过程。

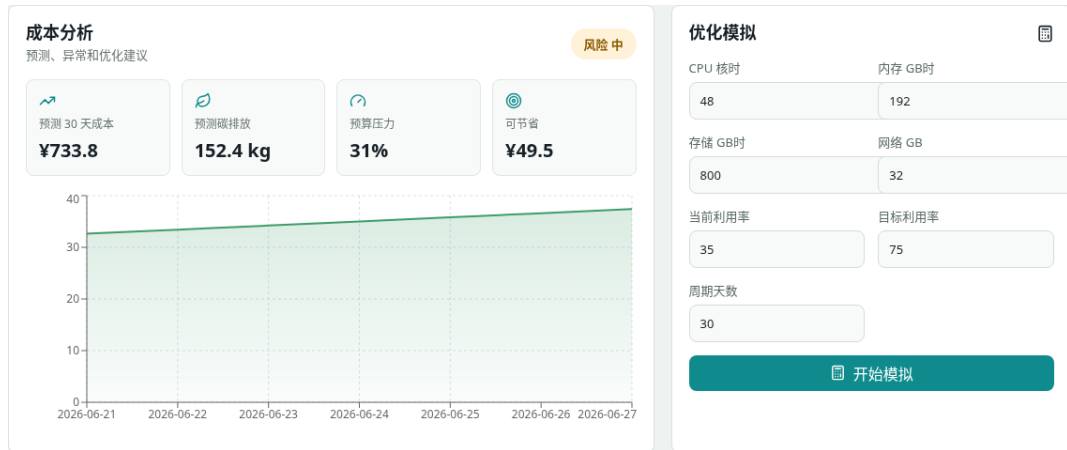


图 2.6 成本分析与优化模拟页面

2.4.5 优化模拟模块

优化模拟模块用于现场演示。用户可以输入当前 CPU、内存、存储、网络和利用率，再设置目标利用率和周期天数。后端会根据同一套成本公式计算优化前后的成本差异和碳减排量。

该功能对应接口：

```
POST /api/insights/simulate
```

我在项目中增加这个模块的原因是：很多系统只能展示已有数据，而优化模拟可以现场改变参数，让老师看到后端确实在实时计算。例如把当前利用率从 35% 调整到 75%，页面会返回预计节省金额和碳减排量，这比静态图表更容易展示系统价值。

2.4.6 告警中心模块

告警中心对应 `cloud_alerts` 表。前端点击“告警”或“执行规则”后，后端会根据以下规则生成告警：

- 未来 30 天预测成本超过总预算 85%。
- 单个资源近 30 天成本超过该资源月预算 80%。
- 成本较高但平均利用率低于 45%。
- 近 30 天碳排放超过阈值。
- 某天成本明显高于历史基线。
- 资源存在但近期没有用量数据。

告警生成后，前端可以点击确认按钮，将告警状态从 `open` 改为 `acknowledged`。这个过程会同时写入审计日志。



图 2.7 告警中心、CSV 导入和运维周报入口

2.4.7 CSV 导入、导出和周报模块

为了让项目更接近实际场景，系统支持 CSV 批量导入用量数据。上传 samples/usage-import-sample.csv 后，后端会读取每一行，根据 resource_id 或 resource_name 匹配资源。如果资源不存在，则自动创建资源，再写入用量记录。

CSV 字段格式如下。

```
resource_name,provider,region,resource_type,owner,monthly_budget,
record_date,cpu_core_hours,memory_gb_hours,storage_gb_hours,network_gb,utilization_score
```

系统还支持下载资源清单 CSV，对应接口为：

```
GET /api/export/resources.csv
```

运维周报功能对应接口为：

```
GET /api/operations/reports/weekly.md
```

后端会根据当前总览、告警和优化建议生成 Markdown 周报，内容包括核心指标、告警概况、重点告警、优化建议、异常成本尖峰和下周行动计划。这个功能可以在演示最后展示，说明系统不仅能录入和查看数据，还能输出管理报告。

2.4.8 Spark 批处理聚合模块

课程任务书中提到建议优先使用 Spark、Flink 等大数据处理框架。因此我在 spark-job/ 中实现了一个 Spark JDBC 批处理任务。该任务读取 cloud_resources 和 usage_records，使用 Spark DataFrame 统计总成本、平均利用率、碳排放、预算风险和成本最高区域，然后写回 analytics_snapshots 表。

运行命令如下。

```
docker compose --profile spark run --rm spark-analytics
```

验证 SQL 如下。

```
select generated_by, total_cost, carbon_kg, snapshot_time
from analytics_snapshots
order by snapshot_time desc
limit 5;
```

本项目已经完成该 Spark JDBC 任务。运行后如果查询结果中出现 generated_by = 'spark' 的记录，就说明 Spark 任务成功读写 openGauss。

2.5 关键实现说明

2.5.1 后端启动自动建表和初始化数据

后端使用 FastAPI 的 lifespan 机制，在服务启动时执行数据库初始化。由于 openGauss 容器首次启动可能需要较长时间，后端会最多重试 30 次数据库连接，每次间隔 2 秒。连接成功后，后端使用 SQLAlchemy 自动建表，并在 AUTO_SEED=true 时写入演示数据。

这个设计解决了一个实际问题：如果后端比数据库更早启动，应用不会立刻失败，而是等待数据库就绪。

2.5.2 openGauss 与 SQLAlchemy 兼容处理

部署时发现 openGauss 虽然兼容 PostgreSQL 协议，但 select version() 返回的版本字符串和 PostgreSQL 不完全一致，SQLAlchemy 解析版本时可能出现异常。项目在 backend/app/db.py 中对 openGauss 版本识别做了兼容处理：如果版本字符串包含 openGauss，就返回一个 SQLAlchemy 可接受的版本号，从而保证 ORM 建表和查询可以正常执行。

这个问题是实验过程中真实遇到的兼容性问题，也是本项目部署部分比较有代表性的排错点。

2.5.3 前端组件拆分

为了避免所有页面逻辑堆在一个文件里，前端将页面拆分为多个组件：

- AppShell.tsx：页面整体布局、侧边栏和顶部操作按钮。
- Metrics.tsx：顶部指标卡。

- OverviewCharts.tsx: 趋势图和资源成本图。
- AnalysisSection.tsx: 成本分析和优化模拟。
- OperationsSection.tsx: 告警、CSV 导入和周报下载。
- ResourceForms.tsx: 资源新增和用量写入表单。
- ResourceTable.tsx: 资源清单、查询、状态切换和删除。
- ActivityPanels.tsx: 预算风险、最近用量和审计日志。

这样划分后，前端代码更接近普通课程项目中的组件化写法，也方便理清楚每个模块的作用。

3 应用部署

3.1 实验环境

本项目部署环境如下。

项目	配置
云平台	华为云开发者空间
操作系统	Linux
容器工具	Docker、Docker Compose v2
数据库	openGauss 容器
后端端口	8000
前端端口	8080
数据库宿主主机端口	15432

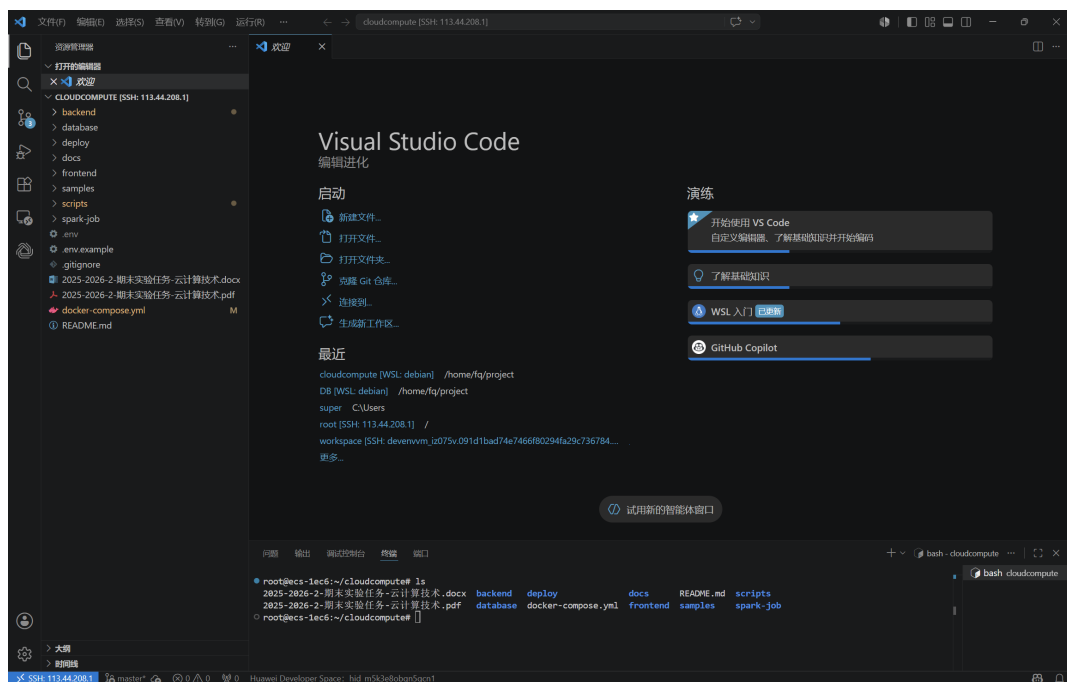


图 3.1 华为云开发者空间项目目录截图

3.2 部署 openGauss / GaussDB

由于实验环境无法稳定直接访问 Docker Hub，我将项目中的镜像改成了国内镜像站。`.env.example` 中主要配置如下。

```
OPENGAUSS_IMAGE=docker.m.daocloud.io/enmotech/opengauss:3.0.0  
PYTHON_IMAGE=docker.m.daocloud.io/library/python:3.13-slim  
NODE_IMAGE=docker.m.daocloud.io/library/node:22-alpine  
NGINX_IMAGE=docker.m.daocloud.io/library/nginx:1.27-alpine  
SPARK_IMAGE=docker.m.daocloud.io/bitnami/spark:3.5  
PIP_INDEX_URL=https://mirrors.aliyun.com/pypi/simple/  
NPM_REGISTRY=https://registry.npmmirror.com
```

部署前先复制环境变量文件。

```
cp .env.example .env
```

单独拉取 openGauss 镜像，验证镜像站可访问。

```
docker pull docker.m.daocloud.io/enmotech/opengauss:3.0.0
```

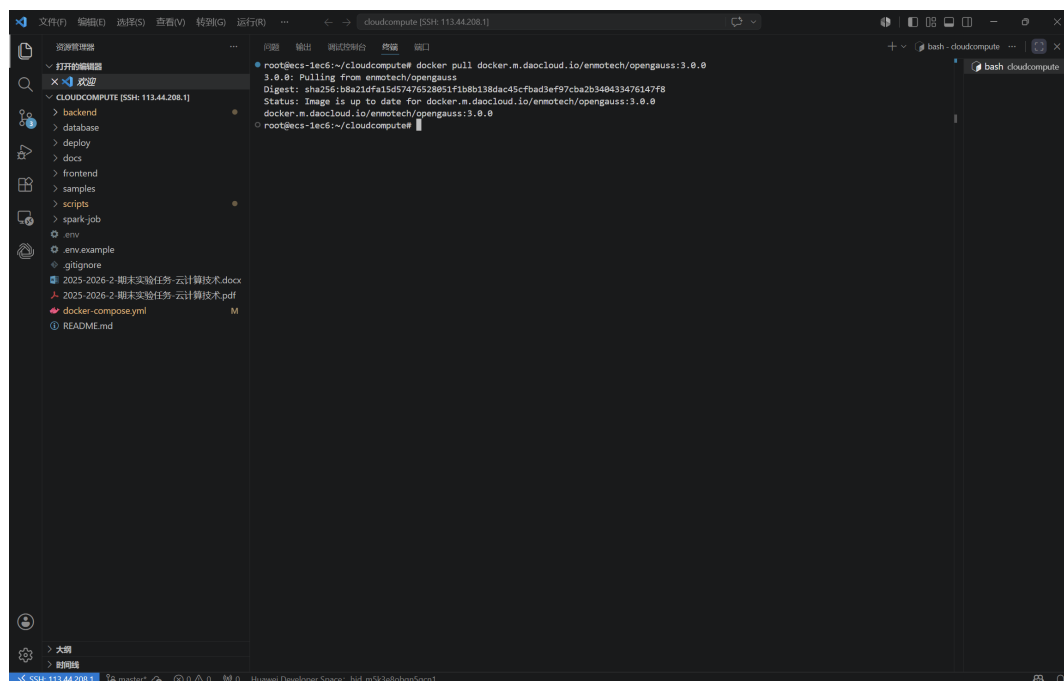


图 3.2 openGauss 镜像拉取成功截图

然后启动所有服务。

```
docker compose up -d --build
```

openGauss 服务在 docker-compose.yml 中的关键配置包括：

- 镜像：docker.m.daocloud.io/enmotech/opengauss:3.0.0
- 容器名：cloudcostlab-opengauss
- 端口映射：15432:5432
- 数据卷：opengauss_data
- 默认密码：CloudGauss@2026
- 健康检查：使用 gsql 执行 select 1

查看容器状态。

```
docker compose ps
```

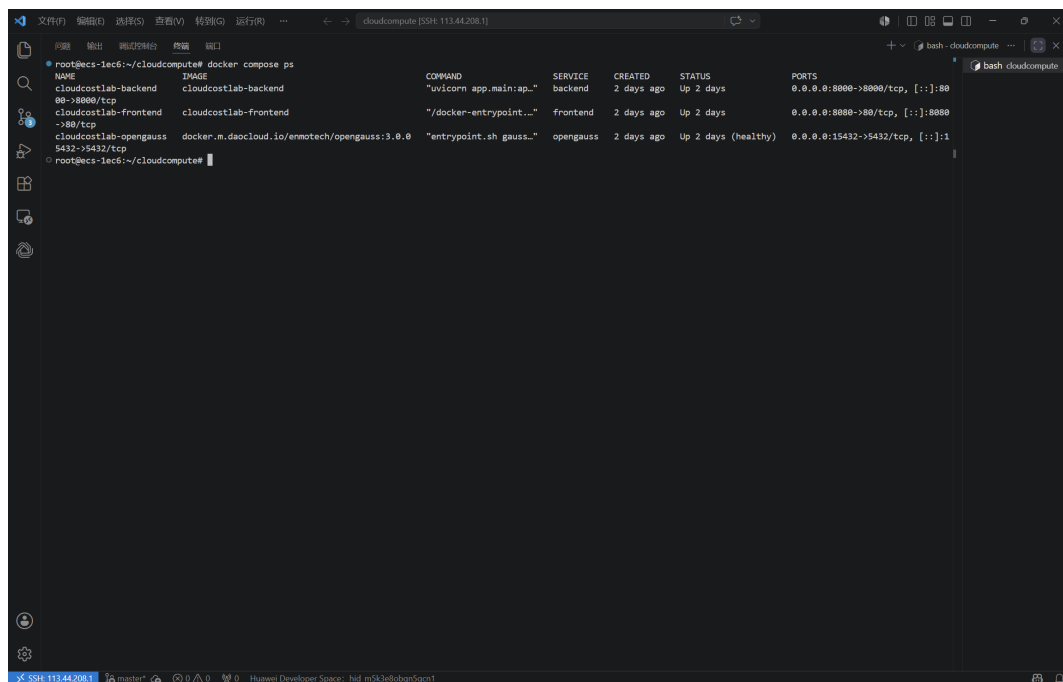


图 3.3 Docker Compose 服务运行状态截图

连接并验证 openGauss。该镜像中 gsql 不在默认 PATH，需要手动补充。如需进入容器排查，可先执行 `docker exec -it cloudcostlab-opengauss bash`，再手动设置 `LD_LIBRARY_PATH` 后调用 `/usr/local/opengauss/bin/gsql`。

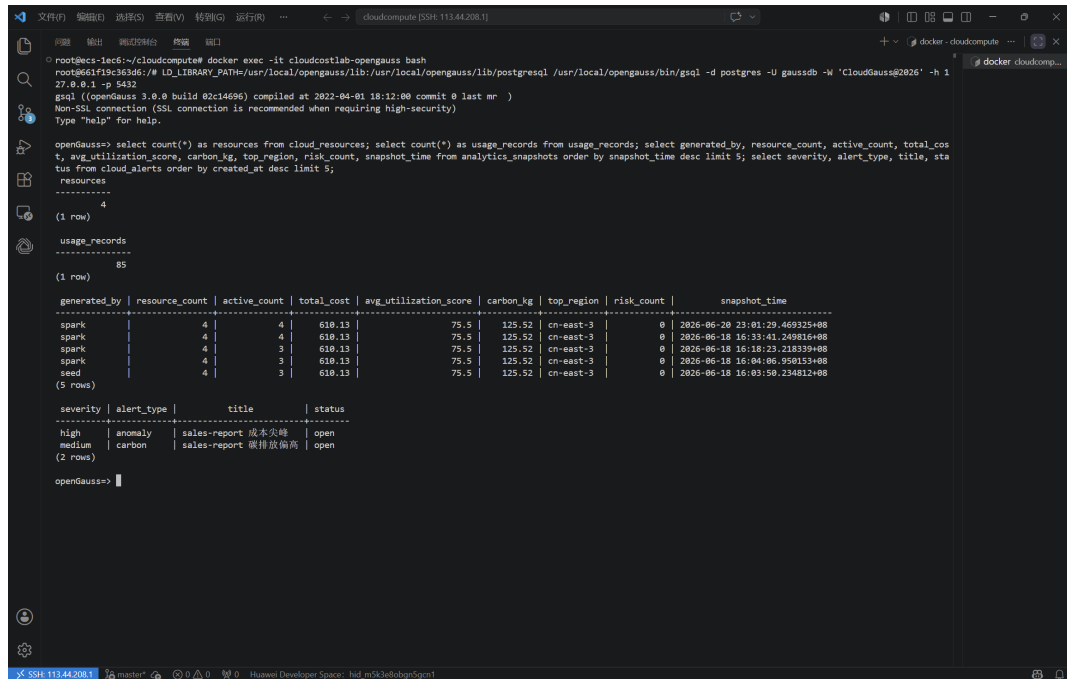


图 3.4 openGauss 数据查询截图

3.3 部署后端应用

后端镜像由 backend/Dockerfile 构建。Dockerfile 的主要流程是：

1. 使用 Python 3.13 slim 镜像。
2. 安装 libpq5，用于数据库连接。
3. 从国内 PyPI 源安装 FastAPI、SQLAlchemy、psycopg2 等依赖。
4. 复制后端代码。
5. 使用 uvicorn 启动服务。

后端容器启动命令由 Docker Compose 管理，实际启动后监听 8000 端口。后端数据库连接串如下。

```
postgresql+psycopg2://gaussdb:CloudGauss%402026@opengauss:5432/postgres
```

其中 opengauss 是 Docker Compose 网络中的数据库服务名，%40 是密码中 @ 的 URL 编码。

后端健康检查接口：

```
curl http://localhost:8000/api/health
```

预期结果：

```
{"message": "ok"}
```

FastAPI 自动生成的接口文档地址：

```
http://localhost:8000/docs
```

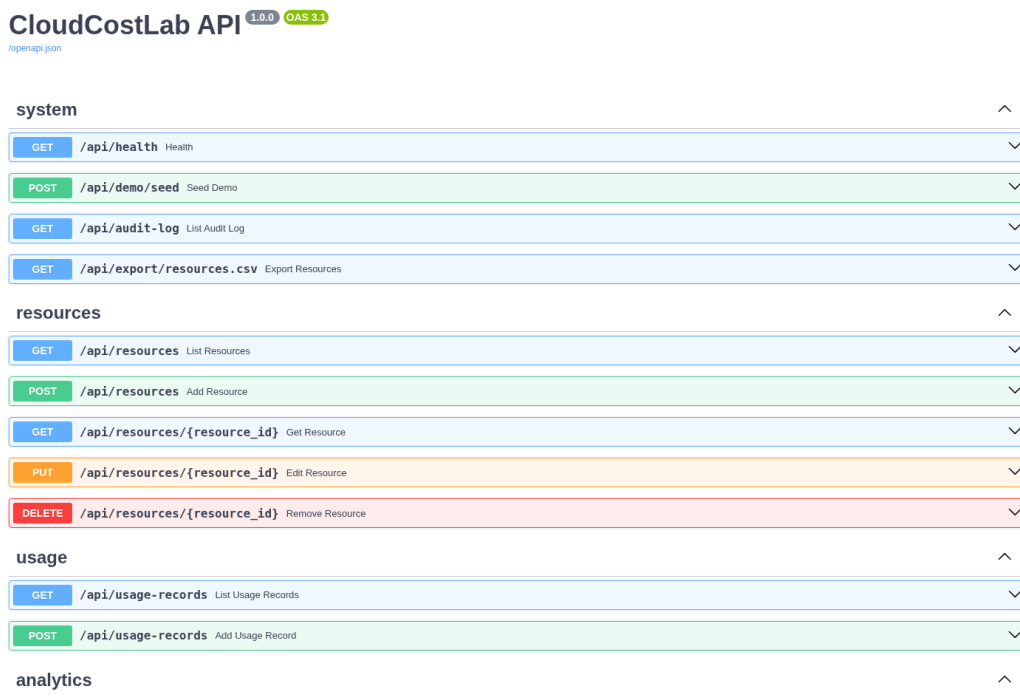


图 3.5 FastAPI Swagger 接口文档

3.4 部署前端应用

前端镜像由 frontend/Dockerfile 构建，采用多阶段构建：

1. 第一阶段使用 Node 镜像安装依赖并执行 `npm run build`。
2. 第二阶段使用 Nginx 镜像作为运行时镜像。
3. 将构建后的 dist 文件复制到 Nginx 静态资源目录。
4. Nginx 将 `/api/` 代理到 `http://backend:8000/api/`。

前端访问地址：

```
http://localhost:8080
```

《大数据与云计算技术》本科生课程实验报告

在华为云开发者空间中，需要将 8080 端口暴露为 Web 访问端口。如果需要展示后端接口文档，也可以同时暴露 8000 端口。

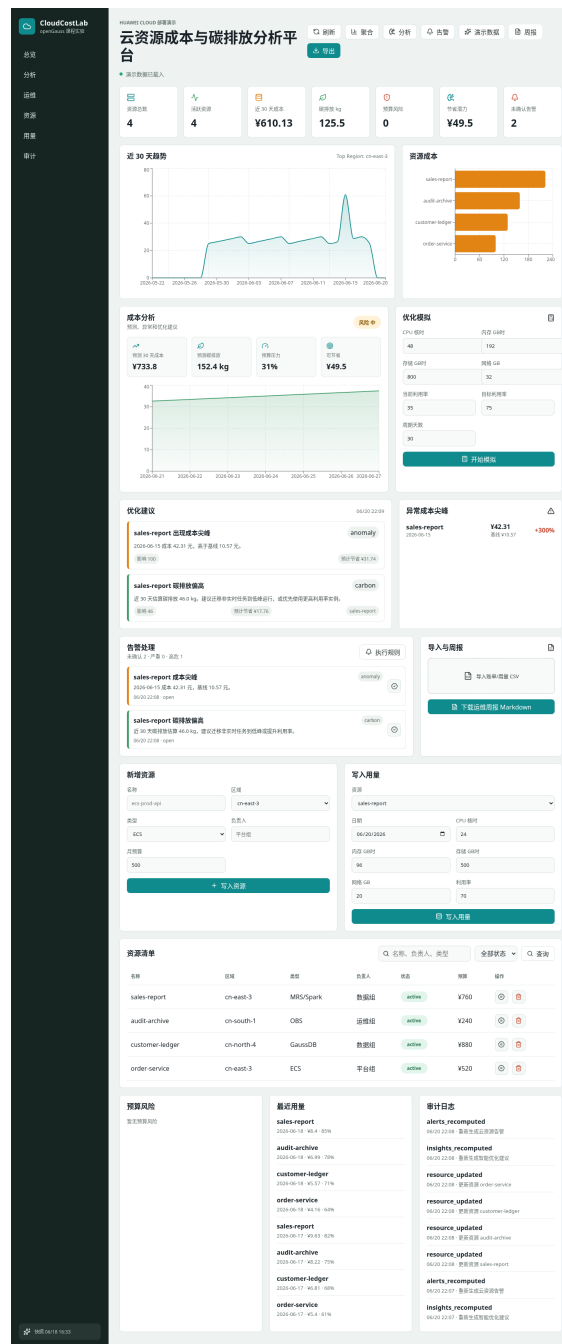


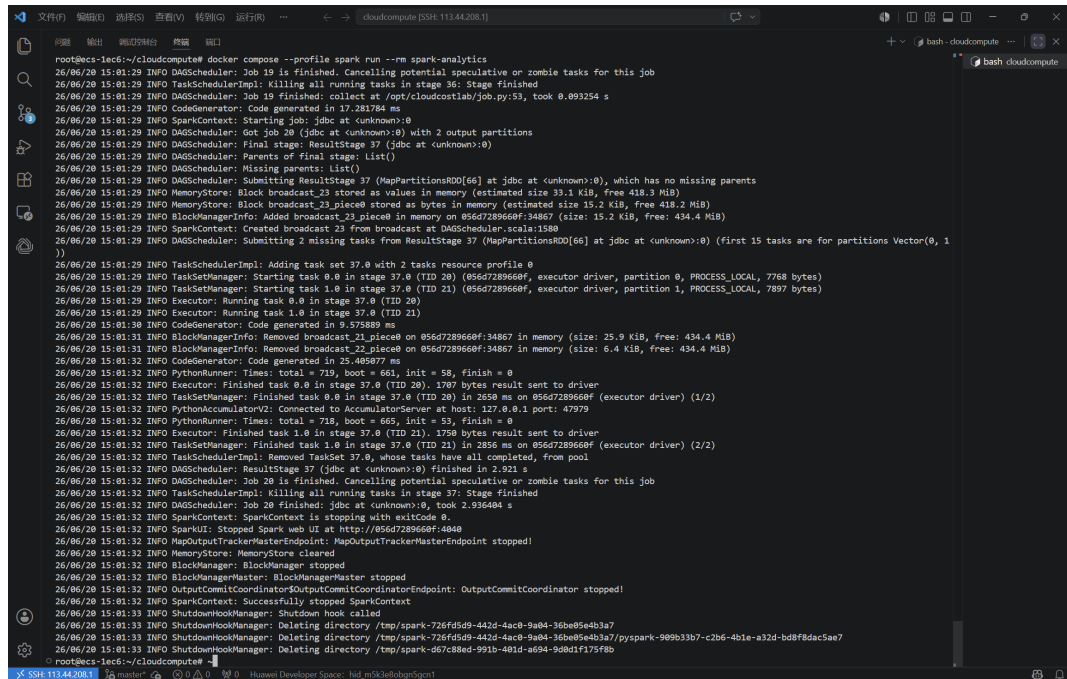
图 3.6 CloudCostLab 前端完整页面

3.5 Spark 批处理任务

Spark 任务使用 Docker Compose profile 管理，不会随主服务自动启动。后端和 openGauss 正常运行后，执行：

```
docker compose --profile spark run --rm spark-analytics
```

《大数据与云计算技术》本科生课程实验报告



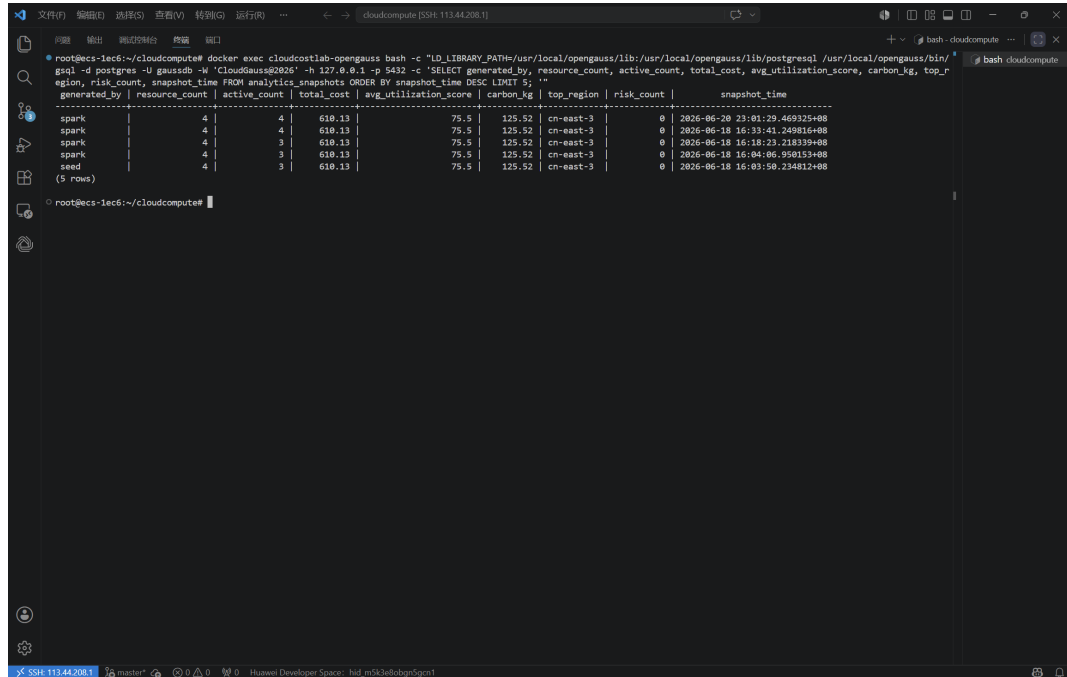
```
root@ecs-1ec6:~/cloudcompute# docker compose --profile spark run --rm spark-analytics
26/06/20 15:01:29 INFO DAGScheduler: Job 19 is finished. Cancelling potential speculative or zombie tasks for this job
26/06/20 15:01:29 INFO TaskSchedulerImpl: Killing all running tasks in stage 36: Stage finished
26/06/20 15:01:29 INFO DAGScheduler: Job 19 finished: collect at /opt/cloudcomputab/job.py:53, took 0.893254 s
26/06/20 15:01:29 INFO CodeGenerator: Code generated in 17.281784 ms
26/06/20 15:01:29 INFO SparkContext: Starting job: jdbc at <unknown>:0
26/06/20 15:01:29 INFO DAGScheduler: Got job 20 (jdbc at <unknown>:0) with 2 output partitions
26/06/20 15:01:29 INFO DAGScheduler: Final stage: ResultStage 37 (jdbc at <unknown>:0)
26/06/20 15:01:29 INFO DAGScheduler: Parents of final stage: List()
26/06/20 15:01:29 INFO DAGScheduler: Missing parents: List()
26/06/20 15:01:29 INFO DAGScheduler: Submitting ResultStage 37 (MapPartitionsRDD[56] at jdbc at <unknown>:0), which has no missing parents
26/06/20 15:01:29 INFO MemoryStore: Block broadcast_23_piece0 stored as values in memory (estimated size 33.1 KiB, free 418.3 MiB)
26/06/20 15:01:29 INFO MemoryStore: Block broadcast_23_piece0 stored as bytes in memory (estimated size 15.2 KiB, free 418.2 MiB)
26/06/20 15:01:29 INFO BlockManagerInfo: Added broadcast_23_piece0 in memory on 056d7289660f:34867 (size: 15.2 KiB, free: 434.4 MiB)
26/06/20 15:01:29 INFO SparkContext: Created broadcast 23 from broadcast at DAGScheduler.scala:1580
26/06/20 15:01:29 INFO DAGScheduler: Submitting 2 missing tasks from ResultStage 37 (MapPartitionsRDD[56] at jdbc at <unknown>:0) (first 15 tasks are for partitions Vector(0, 1))
26/06/20 15:01:29 INFO TaskSchedulerImpl: Adding task set 37.0 with 2 tasks resource profile 0
26/06/20 15:01:29 INFO TaskSetManager: Starting task 0.0 in stage 37.0 (TID 20) (056d7289660f, executor driver, partition 0, PROCESS_LOCAL, 7768 bytes)
26/06/20 15:01:29 INFO TaskSetManager: Starting task 1.0 in stage 37.0 (TID 21) (056d7289660f, executor driver, partition 1, PROCESS_LOCAL, 7897 bytes)
26/06/20 15:01:29 INFO Executor: Running task 0.0 in stage 37.0 (TID 20)
26/06/20 15:01:29 INFO Executor: Running task 1.0 in stage 37.0 (TID 21)
26/06/20 15:01:29 INFO CodeGenerator: Code generated in 9.575889 ms
26/06/20 15:01:31 INFO BlockManagerInfo: Removed broadcast_21_piece0 on 056d7289660f:34867 in memory (size: 25.9 KiB, free: 434.4 MiB)
26/06/20 15:01:31 INFO BlockManagerInfo: Removed broadcast_22_piece0 on 056d7289660f:34867 in memory (size: 6.4 KiB, free: 434.4 MiB)
26/06/20 15:01:32 INFO CodeGenerator: Code generated in 25.405077 ms
26/06/20 15:01:32 INFO PythonRunner: Times: total = 719, boot = 663, init = 58, finish = 0
26/06/20 15:01:32 INFO Executor: Finished task 0.0 in stage 37.0 (TID 20). 1787 bytes result sent to driver
26/06/20 15:01:32 INFO TaskSetManager: Finished task 0.0 in stage 37.0 (TID 20) in 2650 ms on 056d7289660f (executor driver) (1/2)
26/06/20 15:01:32 INFO PythonAccumulatorV2: Connected to AccumulatorServer at host: 127.0.0.1 port: 47979
26/06/20 15:01:32 INFO PythonRunner: Times: total = 718, boot = 665, init = 53, finish = 0
26/06/20 15:01:32 INFO Executor: Finished task 1.0 in stage 37.0 (TID 21). 1798 bytes result sent to driver
26/06/20 15:01:32 INFO TaskSetManager: Finished task 1.0 in stage 37.0 (TID 21) in 2856 ms on 056d7289660f (executor driver) (2/2)
26/06/20 15:01:32 INFO TaskSchedulerImpl: Removed TaskSet 37.0, whose tasks have all completed, from pool
26/06/20 15:01:32 INFO DAGScheduler: ResultStage 37 (jdbc at <unknown>:0) finished in 2.921 s
26/06/20 15:01:32 INFO DAGScheduler: Job 20 is finished. Cancelling potential speculative or zombie tasks for this job
26/06/20 15:01:32 INFO TaskSchedulerImpl: Killing all running tasks in stage 37: Stage finished
26/06/20 15:01:32 INFO DAGScheduler: Job 20 finished: jdbc at <unknown>:0, took 2.936404 s
26/06/20 15:01:32 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/06/20 15:01:32 INFO SparkUI: Stopped Spark web UI at http://056d7289660f:4040
26/06/20 15:01:32 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/06/20 15:01:32 INFO MemoryStore: MemoryStore cleared
26/06/20 15:01:32 INFO BlockManager: BlockManager stopped
26/06/20 15:01:32 INFO BlockManagerMaster: BlockManagerMaster stopped
26/06/20 15:01:32 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/06/20 15:01:32 INFO SparkContext: Successfully stopped SparkContext
26/06/20 15:01:33 INFO ShutdownHookManager: Shutdown hook called
26/06/20 15:01:33 INFO ShutdownHookManager: Deleting directory /tmp/spark-726fd5d9-442d-4ac0-9a04-36be05e4b3a7
26/06/20 15:01:33 INFO ShutdownHookManager: Deleting directory /tmp/spark-726fd5d9-442d-4ac0-9a04-36be05e4b3a7/pyspark-909b33b7-c2b6-4b1e-a32d-bd8f8dac5ae7
26/06/20 15:01:33 INFO ShutdownHookManager: Deleting directory /tmp/spark-d67c88ed-991b-401d-a694-9d0d1f175f8b
root@ecs-1ec6:~/cloudcompute#
```

图 3.7 Spark 批处理任务运行截图

任务执行后，通过以下 SQL 验证：

```
select generated_by, total_cost, carbon_kg, snapshot_time
from analytics_snapshots
order by snapshot_time desc
limit 5;
```

《大数据与云计算技术》本科生课程实验报告



```
root@ecs-1ec6:~/cloudcompute# docker exec cloudcostlab-opengauss bash -c "LD_LIBRARY_PATH=/usr/local/opengauss/lib:/usr/local/opengauss/lib/postgresql/usr/local/opengauss/bin/
gsql -d postgres -U gaussdb -W 'CloudGauss@2026' -h 127.0.0.1 -p 5432 -c 'SELECT generated_by, resource_count, active_count, total_cost, avg_utilization_score, carbon_kg, top_r
egion, risk_count, snapshot_time FROM analytics_snapshots ORDER BY snapshot_time DESC LIMIT 5;'
generated_by | resource_count | active_count | total_cost | avg_utilization_score | carbon_kg | top_region | risk_count | snapshot_time
-----
spark | 4 | 4 | 618.13 | 75.5 | 125.52 | cn-east-3 | 0 | 2026-06-20 23:01:29.469325+08
spark | 4 | 4 | 618.13 | 75.5 | 125.52 | cn-east-3 | 0 | 2026-06-18 16:33:41.249816+08
spark | 4 | 3 | 618.13 | 75.5 | 125.52 | cn-east-3 | 0 | 2026-06-18 16:18:23.218339+08
spark | 4 | 3 | 618.13 | 75.5 | 125.52 | cn-east-3 | 0 | 2026-06-18 16:04:06.950153+08
seed | 4 | 3 | 618.13 | 75.5 | 125.52 | cn-east-3 | 0 | 2026-06-18 16:03:50.234812+08
(5 rows)
```

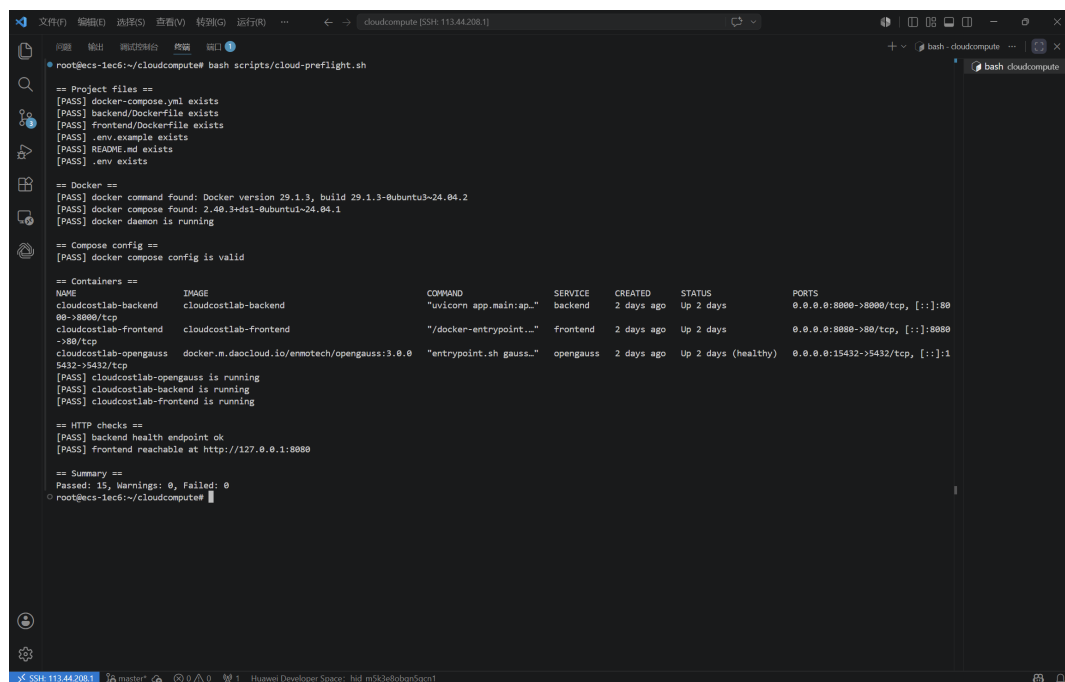
图 3.8 Spark 聚合结果写入 openGauss 截图

4 测试评估

4.1 测试方法

本项目主要从接口、页面、数据库和容器部署四个方面进行验证。接口层面通过 FastAPI 的 Swagger 文档和后端测试用例验证资源、用量、分析、告警和导出接口；页面层面通过浏览器访问华为云开发者空间中暴露的前端地址，检查各个按钮和表单是否能正常触发后端逻辑；数据库层面通过 openGauss 查询确认资源、用量、分析快照、告警和审计日志均已落库；部署层面通过 Docker Compose 状态和访问地址确认前端、后端、数据库容器运行正常。

本地验证时，后端测试用例通过，前端生产构建也能完成。部署验证时，前端页面、后端 Swagger 文档和 /api/health 接口均能正常访问。



```
root@ecs-lec6:~/cloudcompute# bash scripts/cloud-preflight.sh

== Project files ==
[PASS] docker-compose.yml exists
[PASS] backend/Dockerfile exists
[PASS] frontend/Dockerfile exists
[PASS] .env.example exists
[PASS] README.md exists
[PASS] .env exists

== Docker ==
[PASS] docker command found: Docker version 20.10.17, build 10a3c89
[PASS] docker compose found: 2.4.6 34d51-9buntu1-24.04.1
[PASS] docker daemon is running

== Compose config ==
[PASS] docker compose config is valid

== Containers ==


| NAME                   | IMAGE                                        | COMMAND                  | SERVICE   | CREATED    | STATUS              | PORTS                                           |
|------------------------|----------------------------------------------|--------------------------|-----------|------------|---------------------|-------------------------------------------------|
| cloudcostlab-backend   | cloudcostlab-backend                         | "uvicorn app.main:ap..." | backend   | 2 days ago | Up 2 days           | 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp     |
| cloudcostlab-frontend  | cloudcostlab-frontend                        | "/docker-entrypoint..."  | frontend  | 2 days ago | Up 2 days           | 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp         |
| cloudcostlab-opengauss | docker.m.daocloud.io/emotech/opengauss:3.0.0 | "entrypoint.sh gauss..." | opengauss | 2 days ago | Up 2 days (healthy) | 0.0.0.0:15432->15432/tcp, [::]:15432->15432/tcp |


[PASS] cloudcostlab-opengauss is running
[PASS] cloudcostlab-backend is running
[PASS] cloudcostlab-frontend is running

== HTTP checks ==
[PASS] backend health endpoint ok
[PASS] frontend reachable at http://127.0.0.1:8080

== Summary ==
Passed: 15, Warnings: 0, Failed: 0
root@ecs-lec6:~/cloudcompute#
```

图 4.1 scripts/cloud-preflight.sh 自检脚本测试通过

4.2 功能测试结果

功能测试覆盖了课程要求中的主要流程，包括资源管理、用量写入、数据库读写、前后端交互、容器化部署和 Spark 批处理。测试结果汇总如下。

测试类别	验证内容	结果
基础访问	前端首页、后端 Swagger、/api/health	通过
资源管理	新增、查询、状态切换、 删除资源	通过
用量采集	写入 CPU、内存、存储、 网络和利用率数据，自 动计算成本和碳排放	通过
分析功能	生成总览指标、成本趋 势、预算风险、分析快照	通过
成本分析	生成预测、异常成本尖 峰和优化建议	通过
告警中心	生成预算、异常、利用 率、碳排放告警，并支持 确认	通过
导入导出	CSV 用量导入、资源 CSV 导出、Markdown 周 报下载	通过
Spark 批处理	Spark 通过 JDBC 读取 openGauss 明细并写回聚 合快照	通过

4.3 数据库读写验证

数据库验证重点是确认前端操作确实会通过后端写入 openGauss，而不是只停留在页面展示。测试时新增资源、写入用量、执行分析、生成告警后，openGauss 中对应表均能查询到数据变化。其中：

- cloud_resources 能看到资源记录和状态变化。
- usage_records 能看到用量记录、估算成本和碳排放。
- analytics_snapshots 能看到 API 和 Spark 写入的聚合快照。
- cloud_alerts 能看到告警规则执行结果及确认状态。
- audit_events 能看到资源更新、用量写入、分析、告警等审计事件。

预算风险 暂无预算风险	最近用量 sales-report 2026-06-18 · ¥8.4 · 85% audit-archive 2026-06-18 · ¥6.99 · 78% customer-ledger 2026-06-18 · ¥5.57 · 71% order-service 2026-06-18 · ¥4.16 · 64% sales-report 2026-06-17 · ¥9.63 · 82% audit-archive 2026-06-17 · ¥8.22 · 75% customer-ledger 2026-06-17 · ¥6.81 · 68% order-service 2026-06-17 · ¥5.4 · 61%	审计日志 alerts_recomputed 06/20 22:08 · 重新生成云资源告警 insights_recomputed 06/20 22:08 · 重新生成智能优化建议 resource_updated 06/20 22:08 · 更新资源 order-service resource_updated 06/20 22:08 · 更新资源 customer-ledger resource_updated 06/20 22:08 · 更新资源 audit-archive resource_updated 06/20 22:08 · 更新资源 sales-report alerts_recomputed 06/20 22:07 · 重新生成云资源告警 insights_recomputed 06/20 22:07 · 重新生成智能优化建议
-----------------------	---	--

图 4.2 最近用量、预算风险和审计日志

4.4 部署验证

部署后在华为云开发者空间中完成访问验证:前端通过 <http://113.44.208.1:8080> 访问,后端通过 <http://113.44.208.1:8000/docs> 查看接口文档,健康检查接口返回 ok。Docker Compose 中前端、后端和 openGauss 三个核心容器保持运行状态。openGauss 查询使用完整 LD_LIBRARY_PATH + /usr/local/opengauss/bin/gsql 命令执行,可以查询到资源、用量、分析快照和告警数据。

5 总结

5.1 项目开发的挑战与应对方法

本项目开发过程中遇到的第一个问题是 openGauss 容器启动较慢。刚开始后端容器可能比数据库更早启动，导致连接失败。后面我在后端启动流程中加入了数据库连接重试逻辑，等待 openGauss 初始化完成后再建表和写入演示数据。

第二个问题是 openGauss 与 SQLAlchemy 的兼容性。openGauss 使用 PostgreSQL 协议，但版本字符串不完全一样，SQLAlchemy 解析时可能报错。为了解决这个问题，我在数据库初始化代码中增加了 openGauss 版本识别处理，使后端可以正常连接和建表。

第三个问题是镜像拉取。实验环境不能稳定访问 Docker Hub，第一次使用 `docker.m.daocloud.io/opengauss/opengauss:latest` 时还遇到了 403 Forbidden。后来改为 `docker.m.daocloud.io/enmotech/opengauss:3.0.0`，并把 Python、Node、Nginx、Spark 镜像以及 pip、npm、Maven 依赖源都改成国内源，部署过程稳定了很多。

第四个问题是功能较多时页面容易混乱。后来我把前端拆成多个组件，并按“总览、分析、运维、资源、用量、审计”的顺序组织页面，使演示流程更加清楚。

5.2 项目部署存在的问题与不足

本项目目前主要是课程实验级别的实现。openGauss 使用单实例容器部署，虽然满足伪分布式和容器化演示要求，但生产环境中还需要考虑主备高可用、自动备份、密码密钥管理、HTTPS 和权限控制。

成本和碳排放计算目前使用估算公式，适合课程演示和趋势比较，但不能等同于真实云厂商账单。真实环境中应接入华为云账单 API、云监控指标和更准确的区域碳排放因子。

Spark 任务目前是手动运行的批处理任务，没有接入定时调度。如果后续继续完善，可以使用定时任务或工作流工具定期运行 Spark 任务。

另外，前端当前更偏向课程展示型仪表盘，还没有做用户登录、权限划分和多租户隔离。这些功能在真实系统中也比较重要。

5.3 项目展望与学习心得

后续可以从几个方向继续完善。第一，接入华为云真实账单和监控数据，使平台能够分析真实云资源。第二，将 Spark 批处理任务定时化，或者进一步尝试

Flink 流处理。第三，把系统部署到华为云 CCE 或 Kubernetes 中，增加多副本、滚动更新和服务发现。第四，增加用户登录和角色权限，让不同负责人只能查看自己负责的资源。

通过这次实验，我对容器化部署、openGauss 数据库访问、前后端分离开发和云资源分析流程有了更完整的理解。这个项目一开始只是为了满足课程要求，但在逐步实现过程中，我发现只完成“能增删改查”并不能很好地体现云计算课程特点。因此后面加入了成本分析、告警、CSV 导入、周报和 Spark 聚合等功能。最终系统虽然还不能直接用于生产环境，但已经形成了比较完整的数据闭环：前端录入数据，后端计算处理，openGauss 保存结果，页面展示分析，最后还能导出报告。

6 参考文献

- [1] openGauss 官方文档：数据库安装、连接和 SQL 使用说明。
- [2] FastAPI 官方文档：请求处理、依赖注入和自动生成接口文档。
- [3] SQLAlchemy 官方文档：ORM 模型、Session 和数据库连接池。
- [4] React 官方文档：组件、状态和表单处理。
- [5] Vite 官方文档：前端工程构建和生产打包。
- [6] Docker 官方文档：Dockerfile、镜像构建和 Docker Compose。
- [7] Apache Spark 官方文档：Spark SQL、DataFrame 和 JDBC 数据源。
- [8] 华为云开发者空间相关文档：云端开发环境、端口暴露和应用部署。
- [9] 《2025-2026-2 期末实验任务-云计算技术》课程任务书。